



Pontifícia Universidade Católica
do Rio de Janeiro

Trabalho G2

Professor Vitor Pinheiro

INF1022

Victor Hugo Moreira Brito
Rafaela Bessa Garcia de Oliveira

2421278
2420043

O que foi implementado

O trabalho consistiu no desenvolvimento de um analisador sintático e transpilador para a linguagem ObsAct. O sistema recebe um arquivo escrito na linguagem de automação proposta e gera como saída um arquivo equivalente em C.

Requisitos obrigatórios implementados:

- Declaração de dispositivos e identificação de suas respectivas variáveis.
- Geração de código C com inicialização em zero para variáveis de sensores não atribuídas via comando set.
- Suporte às atribuições e estruturas de decisão (se...entao...senao), incluindo operadores lógicos e validações encadeadas com &&.
- Tradução das funções base (ligar, desligar, verificar e variações de alerta) para código C funcional.
- **Envio Broadcast:** Implementação da regra para todos:, permitindo que alertas sejam distribuídos para múltiplos dispositivos simultaneamente.

Funcionalidades adicionais implementadas:

Para expandir as possibilidades de automação, foram inseridas duas novas ações na gramática:

- **Ação fechar:** Comando direto para manipular dispositivos como cortinas e portas.
- **Ação agendar:** Permite passar um horário no formato numérico hh:mm para agendamento de eventos.

Como foi implementado

A solução foi programada em Python utilizando a biblioteca SLY. O pipeline de compilação foi estruturado da seguinte forma:

1. **Análise Léxica (obsact_lexer.py):** Responsável por mapear o arquivo texto em tokens. Além das palavras reservadas e identificadores normais, foram aplicadas expressões regulares para formatar os valores numéricos como inteiros em Python e uma regra específica (`\d{2}:\d{2}`) para reconhecer o token de horário (HORA).
2. **Análise Sintática (obsact_parser.py):** Consome os tokens gerados e monta a Árvore Sintática Abstrata (AST). As reduções das regras gramaticais retornam tuplas estruturadas para manter os dados aninhados (por exemplo, agrupando a condição e os blocos de comandos de uma estrutura if-else).

3. **Transpilação (obsact_transpiler.py):** Recebe a AST e navega recursivamente pelos nós (padrão *Visitor*). O código inicia injetando as bibliotecas em C e declarando as variáveis, mantendo a indentação correta ao descer na árvore. A regra de broadcast funciona iterando sobre a lista de dispositivos retornada na AST e gerando blocos repetidos da função de alerta no arquivo de saída.
-

Gramática Final Utilizada

A gramática final com as adições necessárias. As regras em **VERMELHO** destacam o que foi modificado em relação à especificação original para suportar o recurso de broadcast e as ações extras.

```
PROGRAM    → DEVICES CMDS
DEVICES    → DEVICE DEVICES | DEVICE
DEVICE     → dispositivo: { ID }
           | dispositivo: { ID , ID }
CMDS       → CMD . CMDS | CMD .
CMD        → ATTRIB | OBSACT | ACT
ATTRIB     → set ID = VAR
OBSACT     → se OBS entao CMDS
           | se OBS entao CMDS senao CMDS
OBS        → ID oplogic VAR
           | ID oplogic VAR && OBS
VAR        → NUM | TRUE | FALSE

/* --- ADIÇÕES PARA BROADCAST --- */
LISTA_DEVICES → ID | ID , LISTA_DEVICES

ACT        → ACTION ID
           | enviar alerta ( MSG ) ID
           | enviar alerta ( MSG , ID ) ID
           | enviar alerta ( MSG ) para todos: LISTA_DEVICES
           | agendar ID HORA

ACTION     → ligar | desligar | verificar | fechar
OPLOGIC    → > | < | >= | <= | == | !=
```

O que funciona e o que não funciona

- **O que funciona:** A conversão de sintaxe, incluindo o aninhamento de múltiplos comandos em blocos de decisão, e o mapeamento correto dos nomes de variáveis. O laço de repetição do código de broadcast gera a chamada correta para cada dispositivo. A indentação do C gerado reflete a estrutura da AST, e o código final compila de forma limpa no GCC.
 - **Limitações e o que não funciona:** A recuperação de erro sintático é limitada; um token mal colocado ou a falta de um ponto final levantam um erro (*syntax error*) e interrompem o parser imediatamente, sem tentar recuperar o estado da análise. Na parte de broadcast, a gramática foi simplificada para aceitar apenas mensagens de texto puro, não suportando a concatenação com valores de variáveis nessa chamada específica.
-

Casos de Teste Utilizados

Os testes foram centralizados no arquivo teste.obsact, que aborda diferentes cenários de uso do analisador:

- Inicialização mista (parte explícita pelo comando set e parte implícita via inicialização padrão em 0).
 - Estruturas condicionais complexas avaliando valores com os tokens de lógica dupla (&&) e com múltiplos comandos internos (ex: ligar TV, fechar Cortina).
 - Execução do comando de broadcast iterando sob três dispositivos-alvo.
 - Testes específicos de redução sintática para as novas palavras-chave de ação (fechar e agendar).
-

Como Executar

O ambiente de execução precisa do Python 3.x e da biblioteca de parsing instalada.

1. Instale os requisitos via terminal executando: `pip install sly`
2. No terminal, rode o orquestrador do compilador: `python main.py`
3. Caso a compilação obtenha sucesso, o arquivo de destino `saida.c` será gerado.
4. Para compilar o código fonte traduzido, execute: `gcc saida.c -o saida`
5. Rode o executável gerado: `.\saida.exe`